

CSI247:
Data Structures

Lecture 08 – Searching arrays

Department of Computer Science
B. Gopolang (247-275)



Previous Lesson

- Algorithm growth rate
- Big-O notation

Today …



Learning Objectives

- By the end of this lecture, students must be:
  - Use and implement linear search
  - Use and implement binary search

A scenario

- Imagine the 2025/26 CSI247 classlist from ASAS
- We can traverse this list to perform some operations
- Examples:
  - Check whether student with ID number 202401234 is in the classlist
  - Change name of student with ID 202301723 to "Thomas Jones"
  - Display details of student with ID 202409342

- The above operations require **searching** the classlist

Introduction

- Searching involves **traversing** a collection
  - The process of looking for a specific element in a collection
  - E.g.: checking if a certain student is in the classlist

- Searching is one of the common task in programming
- The traversed collection can be:
  - Ordered (sorted), or
  - unordered



1. Searching

- Why search?
  - To determine if an element is in a collection
  - Or to display/modify some information about an element that is in a collection (or delete item, etc)

- There are several search algorithms
- Of interest in this course:
  - Linear search
  - Binary search

a) Linear Search

- A simple algorithm
- Aka **sequential search**

- Compares search key with each element in the array, sequentially
  - Until search key is found
  - Or we exhaust the collection

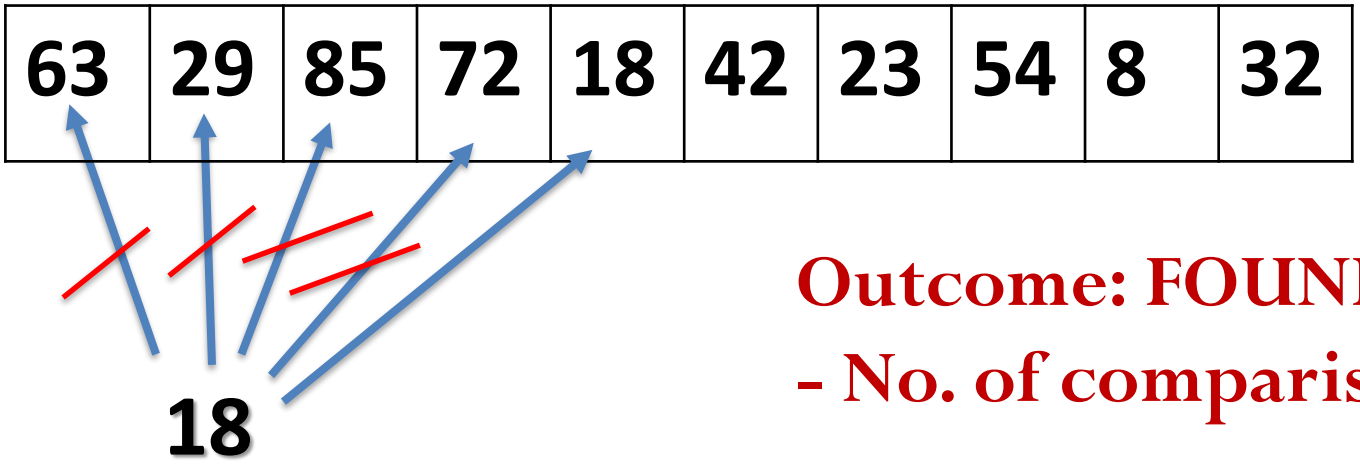
- Algorithm returns:
  - Index of the search element: found
  - -1: Not found:

Note:

- For ordered collection: no need to look at all elements
  - Can stop searching once we determine that the element cannot be in the collection

i) Searching an **unordered** collection

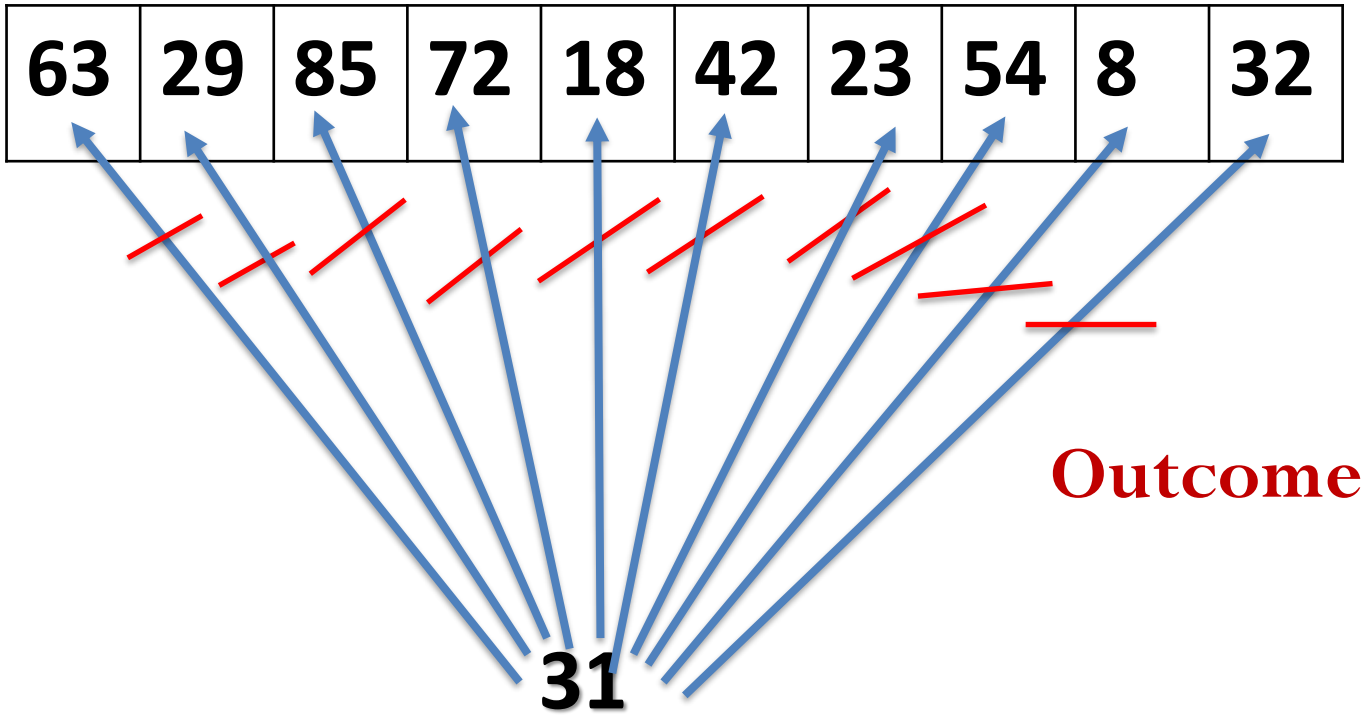
- Searching for 18:



Outcome: FOUND!

- No. of comparisons?

- Searching for 31?



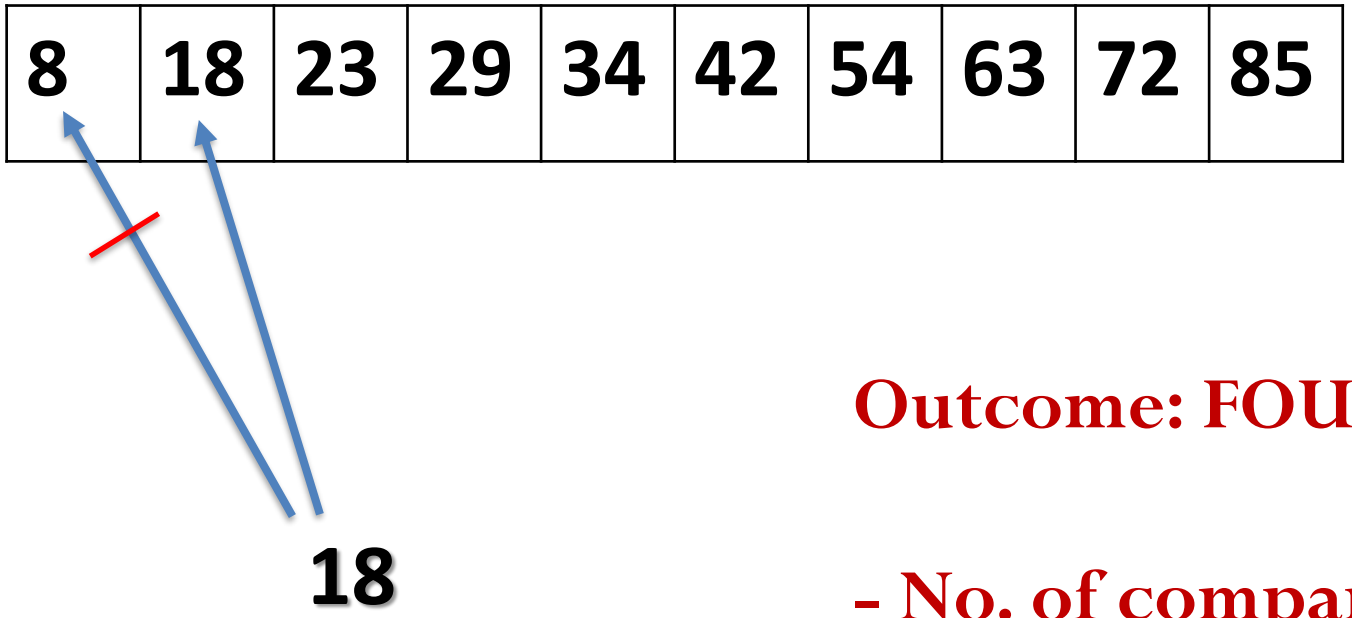
Outcome: NOT FOUND



- No. of comparisons?

ii) Searching an **ordered** collection

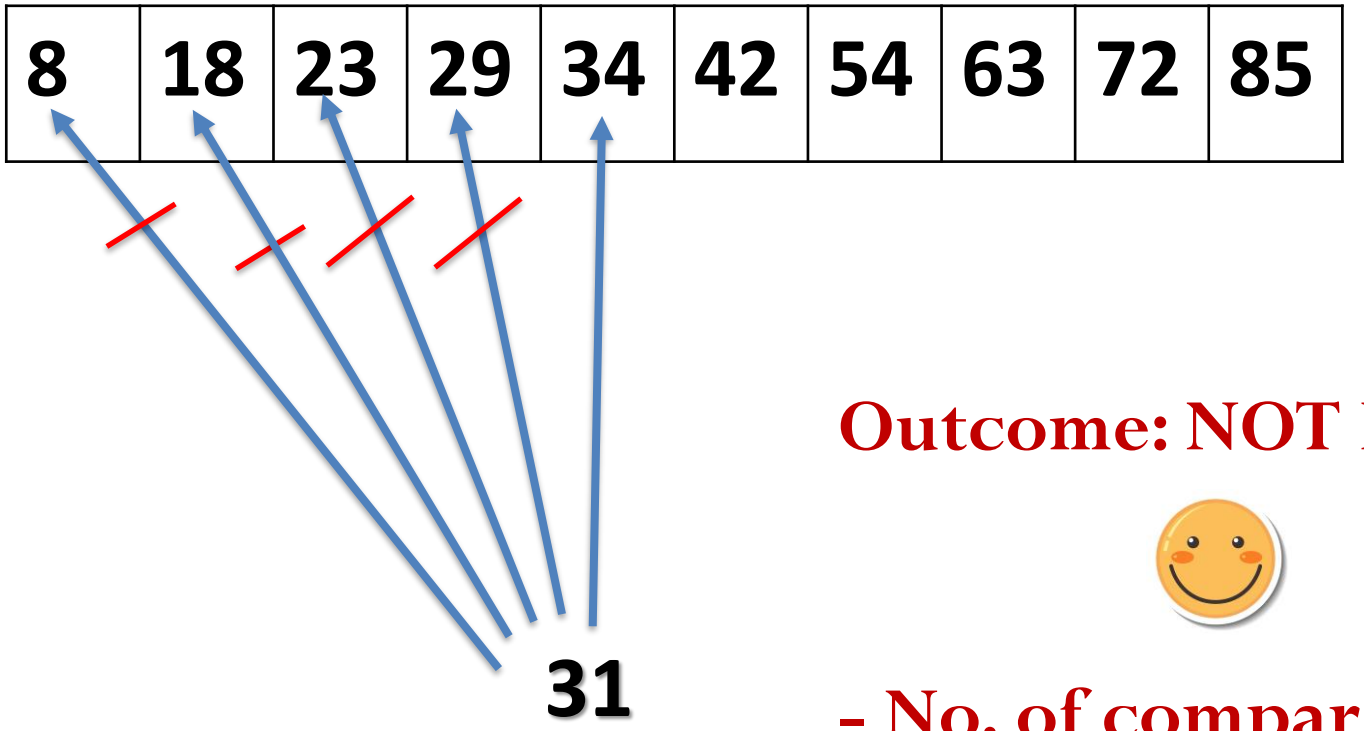
- Searching for 18:



Outcome: FOUND!

- No. of comparisons?

- Searching for 31:



Outcome: NOT FOUND!

- No. of comparisons?

iii) Linear search implementation

- Write an **iterative** method, using linear search algorithm, that accepts an array of integers and an integer value, then return an integer representing the index of the specified value if it is found in the array. Otherwise, the method should return -1.

- Soln?



Sample method

```
public static int linearSearch (int arr[], int key) {
  for (int i = 0;  i < arr.length;  i++) {
    if (arr[i] == key)
      return i;  // found
    }
  return -1; // when key is not found
  }
```

iv) Linear search's time complexity

- Best-case runtime
  - O(1): search item found at index 0
- Worst-case runtime?
  - O(n): search item at index N – 1 or not found

- *Exercise*
  - Convert the iterative method into a recursive one



Observation

- Sorted collection: searching is faster only if the search key is not in the list
  - Collection MUST be sorted

- In general, linear search's **efficiency** is **similar** when searching both unordered and ordered collections



- Application areas
  - For dealing with small collections
  - For handling unsorted data
- Advantages
  - Easy to implement
- Limitations
  - It is not a fast algorithm

- Q:
  - Isn't there a more efficient approach for ordered collections?

- A: There is!
  - We need an approach that will reduce the number of comparisons!
  - Hence more efficient!



b) Binary Search

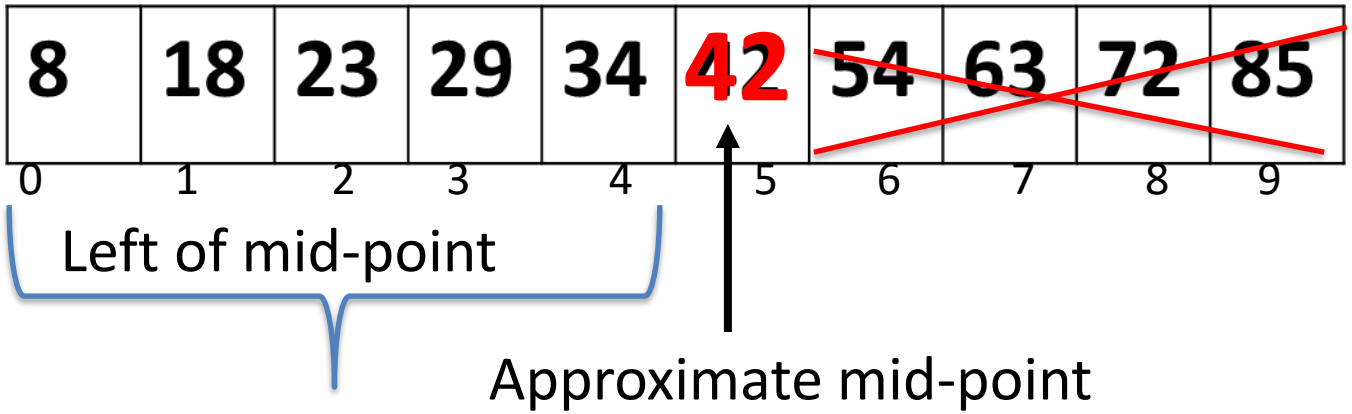
- Requires a sorted collection
- We keep on reducing search space by half
  - Until we find search key
  - OR we exhaust the collection
- Because it uses an ordered (sorted) collection, we stop when we do not find the item where we expect to find it
- Good for data structures that allow for random access (E.g. arrays)

How does Binary Search operate?

- Compare search key with the middle element
- 3 possible cases:
  - **Search key == middle element**
    - Found it, search ends
  - **Search key < middle element**
    - Search the first half (Left) of the array.
  - **Search key > middle element**
    - Search in the second half (Right) of the array.

Illustration

- Search for 18:

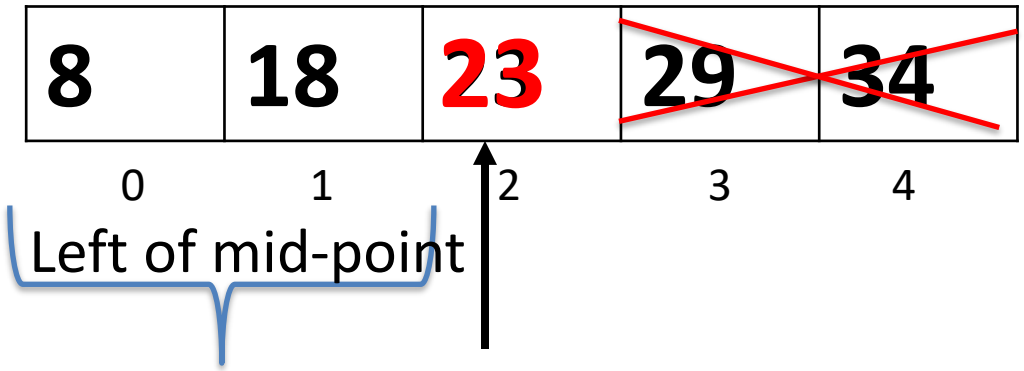


18 == mid-point key? NO.

18 < mid-point key? Yes.

Focus the search on the left side. (Ignore the R side)

--- See next slide ---

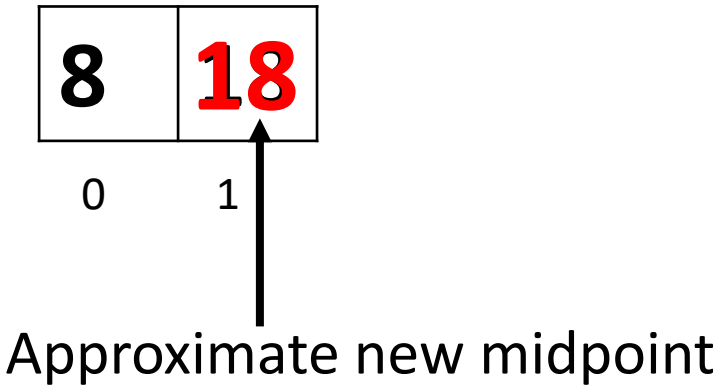


18 == mid-point key? NO.

18 < mid-point key? Yes.

Focus the search on the left side. (Ignore the R side)

--- See next slide ---



18 == midpoint key? YES.

Key found

Stop searching!



Observation

- Binary search reduces the work because it divides the collection into 2

- We need to keep track of which "part" of the array we are focusing on

Binary search implementation

- Write an **iterative** method that accepts an array of int values, search key and the relevant items, then return the index of the search key if found or -1 when not found
- Soln: next slide
- *Exercises*
  - Q: What is binary search's:
    - Best-case runtime?
    - Average-case runtime?
    - Worst-case runtime?
  - Convert the iterative method into a recursive one

Sample method

```
public static int binarySearch (int[] arr, int key, int low, int high) {
        int index = -1; // not yet found the item
        while (low <= high) {
            int mid = low  + ((high - low) / 2);
            if (arr[mid] < key) {
                low = mid + 1;
            } else if (arr[mid] > key) {
                high = mid - 1;
            } else if (arr[mid] == key) {
                index = mid; // found the item
                break;
            }
        }
        return index;
}
```



Summary

- In this lesson, we covered these topics:
  - Linear search
  - Binary search
    - Array MUST be sorted



Next lesson

- Sorting Arrays

Q & A

